



Quiz 09.24 (Note: Paper Quiz. Nothing to upload. Click for Review Guide)

Completion requirements

Due: Wednesday, September 24, 2025, 10:20 AM

DSC 360 — Study Guide for Quiz (≈20 minutes)

What to Expect on the Quiz

The quiz is designed to check your understanding of the **big ideas** we have covered so far. You should be prepared to:

- Explain core concepts such as **tokens**, **embeddings**, **attention**, **softmax**, **temperature**, **fine-tuning**, and **RLHF**.
- Identify the roles of different **layers** in a small **neural network**.
- Work through a simple **pencil-and-paper example** of a feedforward network (with easy numbers).
- Describe, in your own words, how **training** works (forward pass, loss, backpropagation, chain rule, and small weight updates with gradient descent).

The quiz is short (about 20 minutes) and focused on understanding, not memorization of formulas.

Important: Be sure to watch the videos.

1. Machine Learning (ML): The Goal

- We are given **input–output examples** and learn a **parameterized model** that predicts outputs for new inputs.
- **Parameterized** means the model has numbers (**weights**, **biases**) that training adjusts.

2. Neural Networks (NNs): How They Compute

A single unit (neuron)

- Takes real-valued **inputs**, multiplies by **weights**, adds a **bias**, then applies an **activation function**.
- Output is a single **float**. **Hidden-layer** outputs feed forward to the next layer.

Activation functions

- **ReLU**: $\max(0, z)$, used in **hidden layers**.
- **Sigmoid**: squashes to (0,1), useful for **binary classification** outcomes.
- **Softmax**: outputs a **probability distribution** over many **classes**, crucial for **LLMs**.

Layers you should identify

- **Input layer** → **Hidden layer(s)** → **Output layer**
- Be able to look at a tiny **network diagram** and label which nodes belong to each.

3. Training (Backpropagation): How NNs Learn

- A **neural net** is a **parameterized function**; training finds good **weights** and **biases**.
- Example scale: **gemma3:4b** $\approx$ 4 **billion parameters**.

Training loop (conceptual)

- **Feed forward**: compute **outputs** on training data.
- Compare and measure **error**: predicted vs actual → **loss**.
- Send error backward (**backpropagation**):
At the **output layer**, **error** is directly observable.
In **hidden layers**, **error** depends on later-layer errors.
Because the net is a **composite function**, we use the **chain rule** (conceptually) to assign “blame” to each **weight**.
- **Update parameters** with **small adjustments** (tiny steps that reduce **loss**).
- Repeat for many **epochs** until performance stabilizes.

Gradient Descent (and Stochastic Gradient Descent)

- **Gradient descent** is the method we use to adjust the **parameters** (**weights** and **biases**) during training.
- The **gradient** tells us the direction of steepest increase in **loss**; we update parameters in the opposite direction to reduce **loss**.
- We take **small steps** (controlled by the **learning rate**) so the model converges gradually instead of overshooting.
- With large **datasets**, we usually use **stochastic gradient descent (SGD)**: instead of computing updates on the entire **dataset** at once, we update using small random **batches** of data. This makes training faster and often improves **generalization**.

Bottom line: training is an **optimization problem** (minimize **loss** or maximize **likelihood**).

4. Tokens and Embeddings

- **Token**: the basic text unit (**word, subword, punctuation**).
- **Embedding**: a **vector representation** of a **token** (numbers the model can work with).

5. Transformer Architecture (high-level)

- **Input tokens** (plus **positional information** so **order** matters) for **context window**.
- **Embedding layer**: **token IDs** → **embedding vectors**.
- Repeated blocks: **Attention** (models relationships between **tokens**) and **Feed-forward MLP** (nonlinear mixing).
- **Unembedding** (project back to **vocabulary** space) → **Softmax** for next-**token probabilities**.

6. Generation and Temperature

- **Softmax** turns **scores** into **probabilities** for the next **token**.
- **Temperature** controls **randomness**:
Less than 1 → more **deterministic**.
Greater than 1 → more **diverse** and **unpredictable**.

7. How LLMs Are Trained for Completion

- Train on a large **corpus**.
- “Chop” **sequences**; at each step, **predict** the next **token**.
- Compare predicted vs actual → **loss** → **backpropagation** → **optimize parameters**.

8. Fine-Tuning and RLHF

- **Fine-tuning**: additional **training** on a smaller, **domain-specific dataset** to specialize the **base model**.
- **RLHF (Reinforcement Learning from Human Feedback)**: uses **human preferences** to nudge the **model** toward **helpful** and **harmless** behavior.
- **Limitation**: depends on **feedback quality** and is not perfect.

9. Tiny Worked Example (pencil-and-paper)

Network layout: Inputs A, B → Hidden C, D (ReLU) → Output E (identity)

Weights and biases (easy arithmetic):

- $A \rightarrow C: 0.5, B \rightarrow C: -1.0, \mathbf{b_C = 0.0}$
- $A \rightarrow D: 1.0, B \rightarrow D: 0.5, \mathbf{b_D = 1.0}$
- $C \rightarrow E: 2.0, D \rightarrow E: -1.0, \mathbf{b_E = 0.5}$

Given inputs: $A = 2, B = 3$

Hidden pre-activations:

- $\mathbf{C_{in} = (0.5 \cdot 2) + (-1.0 \cdot 3) + 0 = -2 \rightarrow \text{ReLU}(-2) = 0}$
- $\mathbf{D_{in} = (1.0 \cdot 2) + (0.5 \cdot 3) + 1 = 4.5 \rightarrow \text{ReLU}(4.5) = 4.5}$

Output (identity):

- $\mathbf{E_{in} = (2 \cdot 0) + (-1 \cdot 4.5) + 0.5 = -4.0}$
- $\mathbf{E = -4.0}$

Be able to:

- Identify **input**, **hidden**, and **output nodes**.
- Apply **ReLU** correctly.
- Do the simple **arithmetic** to get the final **output**.

10. Final Takeaway

- **LLMs** are **functions**, but they are inherently **unpredictable**.
- Successful apps rely on **testing, testing, testing**: validate with real **examples**, check **edge cases**, and do not assume **consistency** without **evidence**.